Planning

Christoffel's Menu

Nikhil Govender | Student number: 10508831
Subject code: [MAST5112] | Group: [1]

1. Scenario Analysis

Christoffel is a local restaurant owner who needs an app to manage their menu. The chef currently keeps the menu information on paper, which makes it hard to update foods and find information. The Android app will keep this information in one place.

Requirement	Your response
Who will use the application?	The restaurant chef will be the main user. Christoffel, the owner, may also use it to manage the menu.
What is the purpose of the application?	The app will help the chef add, see and edit dishes from a mobile device. This will make the menu easier to organise and update.
What information must the application store?	Each menu item must include a dish name, description, course and price. The course must be Starter, Main Course or Dessert.
List four functions the application should perform.	Add new menu items; view all menu items; edit existing items; delete items that are no longer needed. Editing and deleting will be added in the Final PoE.
List two usability requirements.	Clear navigation: use readable text, clear labels and buttons in the same style. Helpful feedback: explain missing or incorrect information and confirm when an item has been saved.

Development stages

Part 1 will focus on planning and screen designs. Part 2 will allow the chef to add and view menu items and receive validation messages. The final POE will add editing, deleting, searching, filters and menu stats. Each part will build on the work completed before it.

2. User Interface Designs

The design will use green buttons, white backgrounds and dark text. Labels will stay above the fields so that the chef can still see what each field is for after typing. The screens below show the main layout for Part 2. The dishes and prices are examples. Keeping labels visible follows Nielsen's (1994) guidance on reducing what users need to remember.

2.1 Menu and Add Menu Item

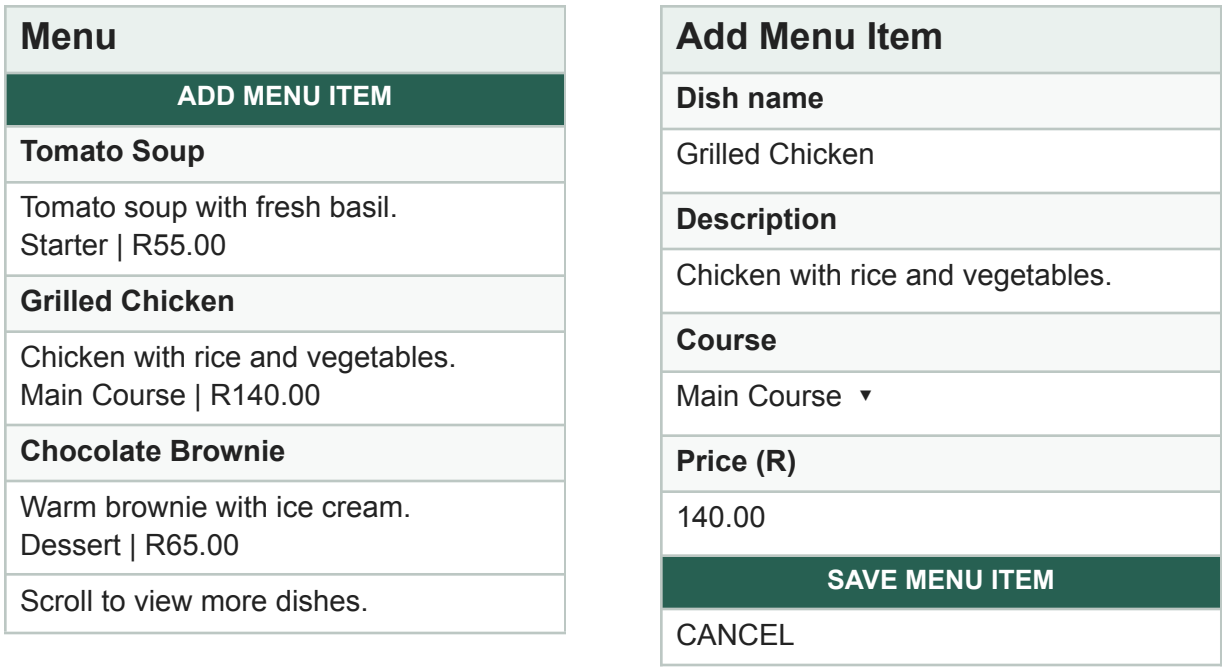


Figure 1: Menu and Add Menu Item screens.

Menu screen: The app will open here and show all saved dishes. Each item will show the dish name, description, course and price. Add Menu Item will open the form.

Add Menu Item screen: The chef will complete the four fields and tap Save Menu Item. If the details are valid, the app will add the dish, return to Menu and show "Menu item saved." Cancel will return without adding an item.

Keyboard and scrolling: The description will allow more than one line. The price field will use a decimal keyboard (React Native, 2026d). The form will scroll when needed so that the keyboard does not hide the Save button.

2.2 Empty Menu, Course Selection and Errors

These are different states of the same Menu and Add Menu Item screens. They show what the chef will see when starting the app or correcting a form.

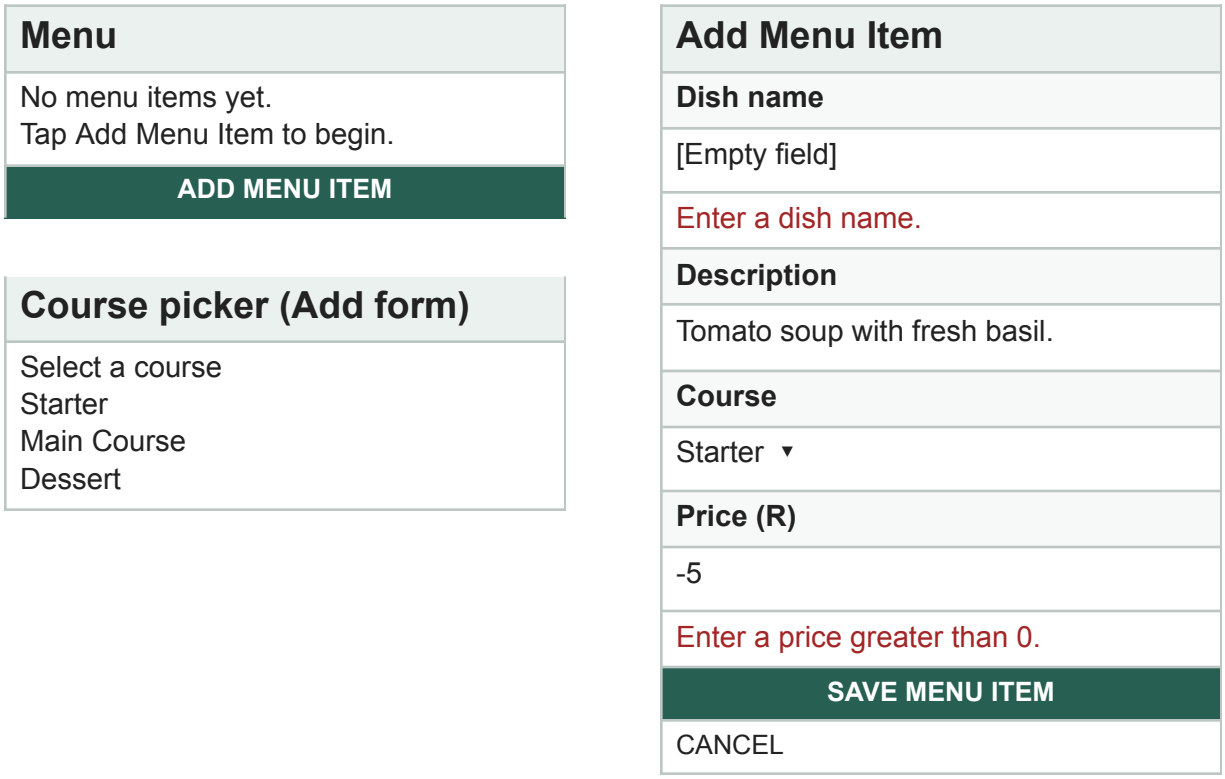


Figure 2: Empty menu, course options and an example of field errors. The course picker belongs to the Add Menu Item form.

Validation and feedback

Check	Message or result
Dish name or description is blank, or contains only spaces.	"Enter a dish name." or "Enter a description."
No course has been selected.	"Select a course."
Price is blank, not a number, zero or negative.	"Enter a valid price greater than 0."
All four fields are valid.	Add the item and show "Menu item saved."

The form will keep the information already entered so that the chef only needs to fix the errors. The initial course prompt will be "Select a course". A price greater than zero is a proposed

validation rule. Nielsen (1994) recommends clear error messages that explain the problem and how to fix it.

2.3 Final PoE Screen Designs

The Final PoE will extend the Menu screen and add an Edit Menu Item screen. The chef will use the same layout and course choices as the Add form.

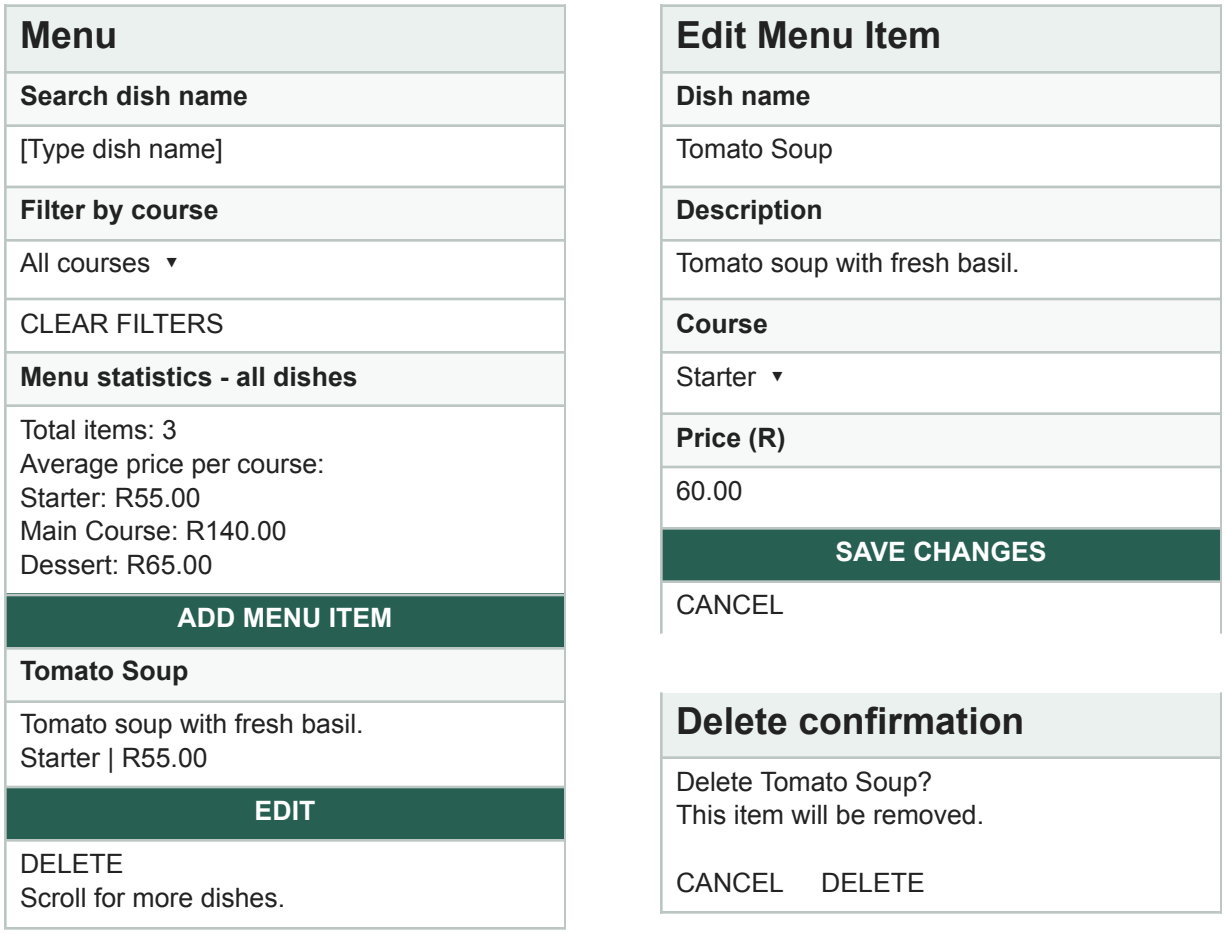


Figure 3: Updated Menu, Edit Menu Item and the delete confirmation. The confirmation opens from Delete on Menu.

Edit and delete: Edit will open the selected dish with its details filled in. Save Changes will use the same validation rules as Add. Cancel will leave the dish unchanged. Delete will ask for confirmation before removing the dish. The Cancel and confirmation options follow Nielsen's (1994) guidance on user control and preventing errors.

Search and filter: Search will match dish names without being affected by capital letters. The filter options will be All courses, Starter, Main Course and Dessert. Search and filter will work together. If nothing matches, show "No matching dishes." Clear Filters will reset both controls.

3. Application Flow Diagram

The app will start on Menu. The first diagram shows the main Part 2 actions. The second shows the planned Final POE actions. Error messages and confirmations are states shown on a screen.

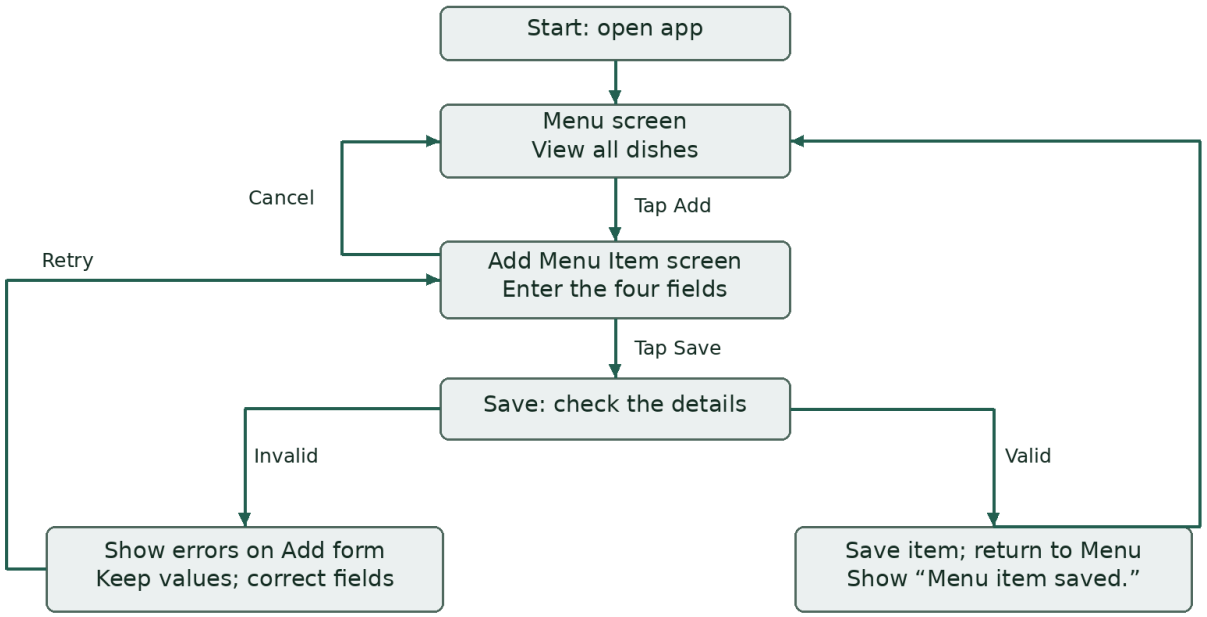


Figure 4: Part 2 flow.

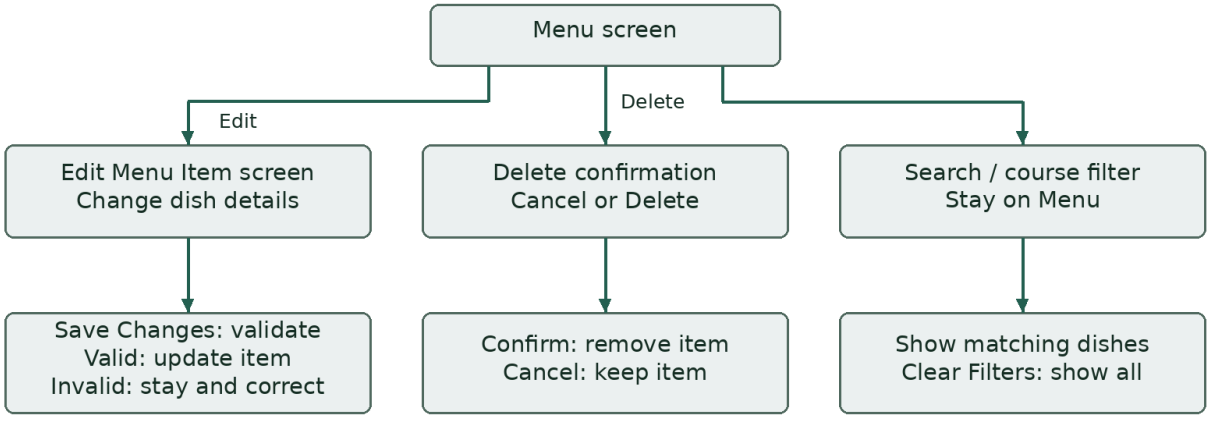


Figure 5: Final PoE flow. Cancel on Edit returns to Menu without saving.

4. React Native Component Planning

The components below match the requirements in the brief. They will be used when the app is built in the later parts.

Application requirement	React Native component	Purpose
Display a heading	Text	Shows headings and labels. Larger text will make screen titles easy to see (React Native, 2026c).
Enter the dish name	TextInput	Allows the chef to type the dish name (React Native, 2026d).
Enter the description	TextInput	Uses multiline input so that the description can cover more than one line (React Native, 2026d).
Select a course	Picker	Lets the chef choose Starter, Main Course or Dessert from fixed options (React Native Picker, n.d.).
Enter the price	TextInput	Uses keyboardType="decimal-pad" for price entry. The app will still check that the price is valid (React Native, 2026d).
Save a menu item	Button	Uses onPress to run the save action. The app checks the fields before adding the dish (React Native, 2026a).
Display menu items	FlatList	Shows saved menu items in a scrolling list. Each item displays the dish details (React Native, 2026b).

Picker package: Picker will come from @react-native-picker/picker, which is installed separately from the React Native core package (React Native Picker, n.d.).

Data and statistics plan

Each menu item will also have its own ID so that edits and deletions affect the correct dish. The app will keep one shared list of items. Prices will be stored as numbers and shown in rand with two decimal places.

The proposed statistics are the total number of dishes and the average price per course. The average will be the total of the prices in a course divided by the number of dishes in that course. A course with no dishes will show "No items". These figures will use the full menu and update after adding, editing or deleting an item.

5. Design Reflection

My app will help Christoffel and the chef manage the restaurant menu from a mobile device. The main goals are to show the dishes, allow new items to be added and make the menu easier to update. Each item will include a dish name, description, course and price. This gives the chef one place to find the information instead of using paper records.

The design will use green buttons, white backgrounds and dark text. Clear headings and labels will make each screen easy to understand. Menu will open first and show the saved dishes. Add Menu Item will open a separate form, with Save and Cancel buttons so that the chef knows how to complete the work.

Text will show headings, TextInput will accept dish information, and FlatList will display the menu in a scrolling list (React Native, 2026b, 2026c, 2026d). The Picker will offer the three course options, which avoids different spellings for the same course (React Native Picker, n.d.). The Button will run the save action when tapped (React Native, 2026a).

The app will check for missing information and incorrect prices before saving. Helpful messages will explain what needs to be fixed (Nielsen, 1994), while keeping the information already entered. A success message will confirm that a dish was saved. Part 2 will focus on adding and viewing dishes. The Final PoE will add editing, deleting, search, course filters and statistics. This keeps the first version manageable and allows more features to be added later.

Reference List

Nielsen, J. (1994). *10 Usability Heuristics for User Interface Design.* Last reviewed 30 January 2024. [online] Available at: <https://www.nngroup.com/articles/ten-usability-heuristics/> [Accessed 5 September 2026].

OpenAI. (2026). *ChatGPT.* [Large language model] Available at: <https://chatgpt.com/> [Accessed 5 September 2026].

React Native. (2026a). *Button.* [online] Available at: <https://reactnative.dev/docs/button> [Accessed 6 September 2026].

React Native. (2026b). *FlatList.* [online] Available at: <https://reactnative.dev/docs/flatlist> [Accessed 6 September 2026].

React Native. (2026c). *Text.* [online] Available at: <https://reactnative.dev/docs/text> [Accessed 8 September 2026].

React Native. (2026d). *TextInput.* [online] Available at: <https://reactnative.dev/docs/textinput> [Accessed 8 September 2026].

React Native Picker. (n.d.). *react-native-picker/picker.* [online] Available at: <https://github.com/react-native-picker/picker> [Accessed 8 September 2026].

AI Disclosure

Disclosure of AI Usage in my Assessment

I used ChatGPT (OpenAI, 2026) to help in this assessment. The places are listed below.

How AI was used: I used ChatGPT to help find sources, check references and format the references.

AI tool used: OpenAI ChatGPT.

Dates used: 7–8 September 2026.

Conversation link: [https://chatgpt.com/share/6aa05073-884c-83ea-a5fd-b3d8b92f061d]